

Gestion de la performance

Comprendre l'event loop, optimiser V8 et tenir la charge

Module 4 / 7

Objectifs du module

- Comprendre le single-thread, l'event loop et l'architecture event-driven
- Écrire du JavaScript performant pour V8, gérer la mémoire et les fuites mémoire
- Traiter les calculs lourds (clusters), profiler et identifier les bottlenecks

Durée : 3h30

Plan du module

1. **Single-thread et event loop** — phases libuv, microtasks/macrotasks, `nextTick` vs `setImmediate`, architecture event-driven (`EventEmitter`)
2. **JavaScript performant pour V8** — Ignition/TurboFan, hidden classes, déoptimisation, mémoire (heap, GC), fuites mémoire
3. **Calculs lourds et clusters** — module `cluster`, profilage (`--prof`, `clinic`, `--inspect`), bottlenecks

1. Single-thread et event loop

Le modèle single-thread

Node.js exécute votre code JavaScript sur **un seul thread**. La concurrence vient de l'**event loop** et du délégation des I/O à libuv (pool de threads).

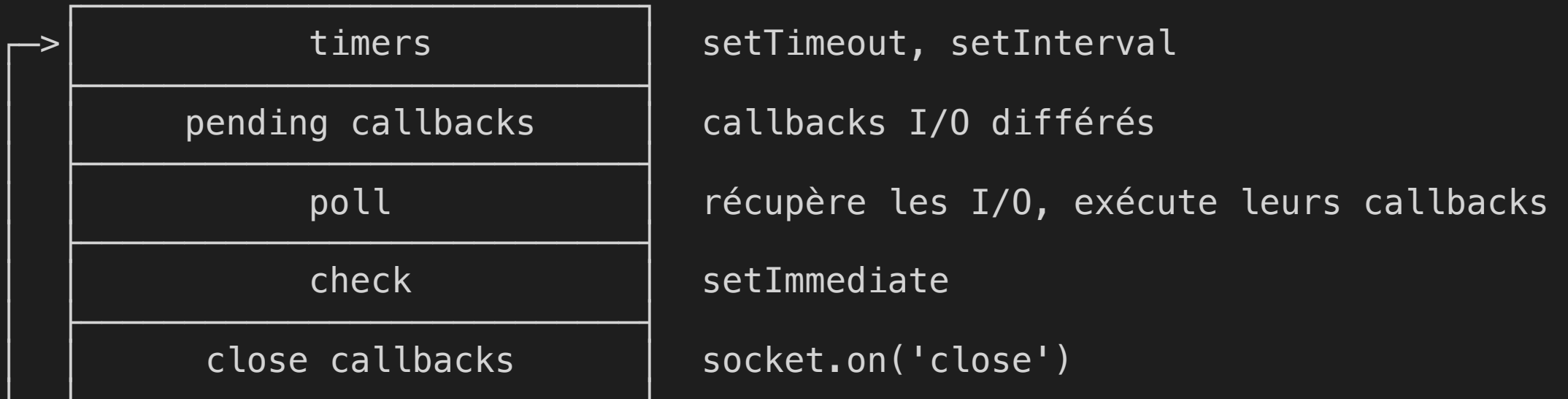
Conséquence clé

- Une opération CPU synchrone **bloque tout** : aucune autre requête n'est traitée pendant ce temps.
- Les I/O (fichier, réseau, DB) sont non bloquantes.

À retenir

- Le code JS n'est jamais préempté.
- Penser « ne jamais bloquer l'event loop ».

Les phases de l'event loop (libuv)



Entre **chaque** phase : vidage de la file des **microtasks**.

Microtasks vs macrotasks

Microtasks (priorité haute)

- `process.nextTick()` (file dédiée, vidée en premier)
- Promesses : `.then` / `await`
- Vidées entièrement avant de passer à la phase suivante

Macrotasks

- `setTimeout`, `setInterval` (phase timers)
- `setImmediate` (phase check)
- I/O (phase poll)

```
console.log('1');
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
console.log('2');
// 1, 2, nextTick, promise, puis timeout/immediate (ordre variable)
```

nextTick vs setImmediate

process.nextTick

- S'exécute **immédiatement après** l'opération courante, avant tout I/O.
- Récursion non contrôlée = **famine** de l'event loop (I/O jamais traités).

setImmediate

- S'exécute à la phase **check**, après les I/O en attente.
- Plus sûr pour « rendre la main » à l'event loop.

```
const fs = require('fs');
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
  // Dans un callback I/O : 'immediate' s'exécute toujours avant 'timeout'
});
```


Architecture event-driven : EventEmitter

La majorité des objets core (streams, serveurs HTTP, sockets) héritent de `EventEmitter` : on s'abonne à des événements plutôt que d'attendre un retour.

```
const { EventEmitter } = require('node:events');

class Commande extends EventEmitter {}
const commande = new Commande();

commande.on('payee', (id) => console.log(`Commande ${id} payée`));
commande.once('expediee', (id) => console.log(`Commande ${id} expédiée`));

commande.emit('payee', 42); // listener appelé
commande.emit('expediee', 42); // once : une seule fois
```

EventEmitter : bonnes pratiques

Pièges

- Fuite d'écouteurs :
`MaxListenersExceededWarning` au-delà de 10 par défaut.
- Un `error` émis sans listener fait **crasher** le process.

Bonnes pratiques

- Toujours gérer `'error'`.
- `removeListener` / `off` pour le cleanup.
- `emitter.setMaxListeners(n)` si justifié, pas par réflexe.

```
emitter.on('error', (err) => logger.error(err)); // évite le crash
```

Démonstration

Reproduction d'une fuite mémoire, capture par heap snapshot, puis scaling avec `cluster`.

Voir `demos/demo-04.md`

2. JavaScript performant pour V8

V8 : Ignition et TurboFan

Ignition (interpréteur)

- Compile le JS en bytecode, exécution immédiate.
- Collecte des informations de type (« feedback »).

TurboFan (compilateur optimisant)

- Recompile en code machine les fonctions « chaudes ».
- S'appuie sur des **suppositions de types** stables.

Du code aux types prévisibles = code optimisé. Des types qui changent = **déoptimisation**.

Hidden classes

V8 associe à chaque objet une **hidden class** décrivant sa structure (forme). Deux objets de même forme partagent la même hidden class : accès aux propriétés ultra-rapide (inline cache).

```
// BON : même forme, hidden class partagée
function Point(x, y) { this.x = x; this.y = y; }
const a = new Point(1, 2);
const b = new Point(3, 4);

// MAUVAIS : ajout de propriété après coup => nouvelle hidden class
const c = new Point(5, 6);
c.z = 7; // forme divergente
// MAUVAIS : ordre d'initialisation différent => autre hidden class
```

Déoptimisation : les causes courantes

Ce qui casse l'optimisation

- Types changeants d'un même champ/argument (`number` puis `string`).
- Ajout/suppression dynamique de propriétés.
- `arguments` détourné, `delete` sur des objets.
- Tableaux « holey » (trous) ou hétérogènes.

Réflexes

- Initialiser tous les champs dans le constructeur, même ordre.
- Fonctions monomorphes (un seul type d'argument).
- Tableaux denses et homogènes.

Gestion de la mémoire : le heap

```
Heap V8
├── New Space (Young Generation)  -> objets récents, GC fréquent (Scavenge)
│   ├── nursery
│   └── intermediate
└── Old Space (Old Generation)    -> objets survivants, GC majeur (Mark-Sweep-Compact)
```

- Limite par défaut de l'old space ~ 2 Go (réglable via `--max-old-space-size`).
- Le **GC est générationnel** : la plupart des objets meurent jeunes.

Le garbage collector

Scavenge (mineur)

- New Space, très rapide, fréquent.
- Copie les survivants, libère le reste.

Mark-Sweep-Compact (majeur)

- Old Space, plus coûteux.
- Marque les objets atteignables, balaie, compacte.

Un GC ne peut **pas** libérer un objet encore référencé. Une référence oubliée = mémoire qui ne redescend jamais = **fuite**.

Les fuites mémoire typiques

```
// 1. Cache global non borné
const cache = {};
app.get('/produit/:id', (req, res) => {
  cache[req.params.id] = lourdObjet(); // grossit sans fin
  res.json(cache[req.params.id]);
});

// 2. Listeners ajoutés sans retrait
setInterval(() => emitter.on('tick', handler), 1000);

// 3. Closures retenant de gros contextes
// 4. Timers jamais nettoyés
```

Signature : heap qui **monte en escalier** sans jamais redescendre après GC.

Détecter et corriger les fuites

Détecter

- `process.memoryUsage()` (heapUsed, rss).
- Heap snapshots (`--inspect` + DevTools), comparer 2 instantanés.
- Marqueur « Detached » / objets retenus.

Corriger

- Borner les caches : `lru-cache`, TTL.
- `Map` / `Set` -> `WeakMap` / `WeakRef` quand pertinent.
- Retirer les listeners et `clearInterval`.

```
const v8 = require('node:v8');  
v8.writeHeapSnapshot('./snap.heapsnapshot'); // capture programmatique
```

3. Calculs lourds et clusters

Le problème : CPU-bound bloque l'event loop

```
// Calcul synchrone : pendant son exécution, AUCUNE autre requête
app.get('/hash', (req, res) => {
  let resultat = req.query.data;
  for (let i = 0; i < 1e7; i++) {           // boucle lourde synchrone
    resultat = require('crypto')
      .createHash('sha256').update(resultat).digest('hex');
  }
  res.json({ resultat });
});
```

Tant que cette route tourne, l'event loop est figé : les autres clients attendent.

Le module cluster

`cluster` fork le process maître en plusieurs **workers** partageant le même port. Chaque worker a son propre event loop = on exploite tous les cœurs CPU.

```
const cluster = require('node:cluster');
const os = require('node:os');

if (cluster.isPrimary) {
  for (let i = 0; i < os.availableParallelism(); i++) cluster.fork();
  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} mort, relance`);
    cluster.fork();
  });
} else {
  require('./server'); // chaque worker lance l'app Express
}
```

cluster, worker_threads ou process externe ?

cluster

- Multi-process, isole les pannes, scale les requêtes I/O et CPU.
- Pas de mémoire partagée (chaque worker = heap séparé).

worker_threads

- Threads dans le même process, mémoire partageable (`SharedArrayBuffer`).
- Idéal pour un calcul CPU ponctuel sans dupliquer l'app.

Règle simple : `cluster` pour scaler une API, `worker_threads` pour déporter un calcul lourd unitaire.

Profilage : --prof

```
node --prof server.js          # génère isolate-*.log  
# ... générer de la charge ...  
node --prof-process isolate-*.log > profil.txt
```

Le rapport classe le temps CPU par fonction (ticks), distingue JS, C++ et GC. On repère les fonctions « **chaudes** » qui dominent le temps d'exécution.

Profilage : clinic et --inspect

clinic.js

- `clinic doctor` : diagnostic global (event loop, CPU, mémoire).
- `clinic flame` : flamegraph du CPU.
- `clinic bubbleprof` : asynchronisme.

--inspect

- `node --inspect server.js` puis DevTools.
- CPU profiler, heap snapshots, allocation timeline.

```
npx clinic doctor -- node server.js
```

Identifier les bottlenecks

Symptôme	Cause probable	Outil
Latence qui explose sous charge	Event loop bloqué (CPU sync)	clinic doctor, flame
RSS qui monte sans redescendre	Fuite mémoire	heap snapshots
GC fréquent / pauses	Allocations excessives	<code>--trace-gc</code>
1 cœur saturé, autres oisifs	Pas de cluster	<code>cluster</code> / load test

Mesurer le débit réel : `autocannon -c 100 -d 30 http://localhost:3000/route`.

Travaux pratiques

Profiler une API, identifier et corriger une fuite mémoire, scaler un calcul lourd CPU-bound avec `cluster`, et mesurer les débits avec `autocannon`.

Fichier : `tp/tp-04.md`

Durée estimée : 1h30 — Difficulté 4/5

Récapitulatif

- L'event loop est single-thread : **ne jamais le bloquer** ; comprendre phases, micro/macrotasks, `nextTick` / `setImmediate` .
- V8 optimise le code aux types stables (hidden classes) ; surveiller le heap et borner ce qui grossit pour éviter les fuites.
- Pour le CPU-bound : `cluster` / `worker_threads` , et **profiler** (`--prof` , `clinic` , `--inspect`) avant d'optimiser.

Cheatsheet : `cheatsheet/module-04.md`

Questions ?

Module suivant : Gestion avancée des flux Node.js
